

SISTEMA E-COMMERCE

Guia Didatico da Terceira Entrega Implantacao no PostgreSQL + CRUD + DAO + JOIN/GROUP BY

Objetivo deste PDF:

explicar o funcionamento do sistema com foco direto no que foi pedido pelo professor.

Arquivos principais analisados:

- implementacao/database.sql
- implementacao/ecommerce-api/dao/*.js
- implementacao/ecommerce-api/server.js
- implementacao/ecommerce-api/frontend/*

Resumo rapido:

- 1) Banco modelado em PostgreSQL com constraints, trigger e integridade.
- 2) DAO implementado para Cliente, Categoria, Produto e Pedido.
- 3) API Express com CRUD completo e frontend modular para operacao.
- 4) JOIN e GROUP BY implementados nos relatorios de pedidos e vendas.

1) Requisitos do professor (itens 6, 7, 8 e 9)

Item 6: criar script no PostgreSQL + desenvolver aplicacao com CRUD.

Item 7: entregar script da base, DAO e aplicacao CRUD via DAO.

Item 8: usar ao menos JOIN ou GROUP BY nas consultas.

Item 9: linguagem/plataforma de livre escolha.

Como o sistema atende cada item

- Script PostgreSQL: implementacao/database.sql
contem CREATE TABLE, constraints, trigger e funcao de validacao.
- DAO no projeto: pasta implementacao/ecommerce-api/dao
ClienteDAO, CategoriaDAO, ProdutoDAO, PedidoDAO e SchemaDAO.
- Aplicacao CRUD via DAO: implementacao/ecommerce-api/server.js
endpoints REST de create, read, update e delete.
- JOIN/GROUP BY presentes:
PedidoDAO.listarPedidosComCliente() usa JOIN com cliente/pf/pj/item.
PedidoDAO.totalVendasPorCliente() usa agregacao com GROUP BY.
- Plataforma:
backend Node.js + Express, banco PostgreSQL, frontend HTML/CSS/JS.

2) Banco de dados PostgreSQL (script)

Arquivo: implementacao/database.sql

Tabelas principais:

cliente, endereco, telefone, pessoa_fisica, pessoa_juridica, categoria, produto, pedido, item_pedido e status_entrega.

Regras de integridade implementadas

- PK e FK em todas as relacoes principais.
- CHECK em tipo de cliente (PF/PJ).
- CHECK em genero (M/F).
- CHECK em quantidade/preco/desconto do item_pedido.
- UNIQUE em email do cliente e em telefone por cliente.
- Trigger + funcao PL/pgSQL:
bloqueia item_pedido quando produto estiver indisponivel.

Mini visao relacional (texto)

```
cliente (1) -> (N) pedido
pedido (1) -> (N) item_pedido
produto (1) -> (N) item_pedido
categoria (1) -> (N) produto
cliente (1) -> (1) endereco
cliente (1) -> (N) telefone
cliente (1) -> (0..1) pessoa_fisica / pessoa_juridica
```

3) Arquitetura da aplicacao



Responsabilidades por camada:

- Frontend: formulario, navegacao, chamadas fetch e render de tabelas/relatorios.
- API: valida entrada, chama DAO e devolve JSON padronizado.
- DAO: concentra SQL (CRUD, JOIN, GROUP BY e transacoes).
- Banco: constraints, FKs, checks e trigger de disponibilidade.

4) CRUD através do DAO

Arquivo de rotas: implementacao/ecommerce-api/server.js

Entidades com CRUD completo

- /clientes
- /categorias
- /produtos
- /pedidos

Exemplo didático de fluxo (create pedido)

- 1) Front envia pedido com id_cliente e itens.
- 2) PedidoDAO valida lista de itens e quantidade.
- 3) PedidoDAO consulta produto para buscar preco_base atual.
- 4) PedidoDAO bloqueia se produto indisponível.
- 5) Transacao: grava pedido e depois grava item_pedido.
- 6) Retorna pedido criado em JSON.

Observação técnica importante

Preço do item não é digitado no pedido.
Ele é buscado automaticamente da tabela produto (preco_base),
garantindo coerência com o estado atual do catálogo.

5) Requisito de JOIN e GROUP BY

JOIN (relatorio de pedidos com cliente):

metodo PedidoDAO.listarPedidosComCliente()

junta pedido + cliente + pessoa_fisica/pessoa_juridica + item_pedido.

GROUP BY (total de vendas por cliente):

metodo PedidoDAO.totalVendasPorCliente()

soma total por cliente e conta quantidade de pedidos.

Trecho simplificado de SQL

```
SELECT c.id_cliente, ccn.nome, COUNT(tpp.id_pedido), SUM(tpp.total_pedido)
FROM cliente_com_nome ccn
LEFT JOIN total_por_pedido tpp ON tpp.id_cliente = ccn.id_cliente
GROUP BY ccn.id_cliente, ccn.nome, ccn.tipo;
```

Resultado esperado no sistema:

- Tela de relatorios mostra pedidos com dados do cliente.
- Tela de relatorios mostra total de pedidos e total de vendas por cliente.
- Esses dados atendem diretamente o item 8 da atividade.

6) Como executar e apresentar

Passo a passo minimo

- 1) Criar banco no PostgreSQL (ex.: ecommerce).
- 2) Executar implementacao/database.sql.
- 3) Ajustar credenciais em implementacao/ecommerce-api/db.js.
- 4) Instalar dependencias: npm install
- 5) Subir API: node server.js
- 6) Abrir frontend: implementacao/ecommerce-api/frontend/index.html

Checklist para a apresentacao da entrega

- [] Mostrar script SQL e principais tabelas/relacoes.
- [] Mostrar CRUD de cliente, produto e pedido funcionando.
- [] Mostrar validacao de produto indisponivel no pedido.
- [] Mostrar endpoint/tela de JOIN (pedidos com cliente).
- [] Mostrar endpoint/tela de GROUP BY (vendas por cliente).
- [] Explicar onde o DAO centraliza regras e SQL.

Conclusao

A implementacao atende os itens da Terceira Entrega:
script PostgreSQL + DAO + aplicacao CRUD com JOIN/GROUP BY.
A arquitetura esta organizada e pronta para demonstracao em sala.